SSA Report

04.02.2026

Roberto Di Rosa

Contents

1. Introduction

How do we trust smart contracts to be sound? The answer lies in using the echidna, this dynamic testing program allows smart contracts developers to verify the soundness of their code. It works by fuzzing input to the specified contract this will test the invariants specified in such that it wants to break them the more function a contract has the bigger the explorable space will become, this means that testing all possible inputs with all possible combinations is actually quite complex(It's not terminable). The main problem with this tool is that echidna itself is not formally verified therefore even if the tests pass it does not guarantee that the contract is actually sound. So should we trust in echidna and echidna alone, No we should see that the code follows safe patterns, and that the invariants we wrote are actually correct.

```
echidna --test-mode assertion test/testtaxpayer.sol --corpus-dir corpus_dir
```

Listing 1: Echidna cmd example for testing the taxpayer invariants

```
call TimeTest::test_blocks_forward(uint256)(2) <no source map>
 ├ call HEVM::roll(4422819) (/home/rob/gits/sec_proj/src/Time.sol:23)
 │  └ ← 0x
 └ ← 0x
call 0xbd748c717Ca30862991DC44AAf0B4db9F8DE12dA::getLotteryWins() <no source map>
 └ ← (1)
call 0x70BEce5a3D1a6eFBC54e1A134cfF3b47EF346bbE::getLotteryWins() <no source map>
 └ ← (4)
emit AssertionFailed(reason=«The lottery is unfair, Expected val:40 Approx:58 Delta:40») <no source map>
```

Figure 1: Example of echidna showing an AssertionFailed event being triggered for the testing the lottery

```
AssertionFailed(..): passing

Unique instructions: 2329
Unique codehashes: 2
Corpus size: 10
Seed: 5102115559004133452
Total calls: 50201

WARNING: assertion at file: src/Time.sol starting at line: 19 was never reached
[2026-02-04 15:17:04.86] Saving test reproducers... Done! (0.000002863s)
[2026-02-04 15:17:04.86] Saving corpus... Done! (0.000309602s)
[2026-02-04 15:17:04.86] Saving foundry reproducers... Done! (0.00001187s)
[2026-02-04 15:17:04.86] Saving coverage... Done! (0.079160762s)
```

Figure 2: Example of echidna passing for every test in testtaxpayer

1.1. The Tax-Payer problem

The first requirement we need to check is if the smart contract written inside of Taxpayer.sol actually enforces sane Taxpayer constraints. We define the correct state using the following invariants:

- **Two way marriage**: If t_1 is married to t_2, then t_2 must be married to t_1[cite: 15].

- $\neg\exists t_1, t_2 : \text{Taxpayer}(t_1) \land \text{Taxpayer}(t_2) \land \text{married}(t_1, t_2, \text{True}) \land \text{married}(t_2, t_1, \text{False})$

- **A Taxpayer's tax allowance cannot be greater than his pool allowance**: A basic sanity check to ensure individual allowances are subsets of the household pool.
  - $\neg\exists t, p, a : \text{Taxpayer}(t) \land \text{get\_tax\_allowance}(t, a) \land \text{get\_pool\_tax\_allowance}(t, p) \land a > p$

- **Married Taxpayers Pooling Consistency**: Married couples must share the exact same pool allowance value. Furthermore, the sum of their individual allowances must equal that shared pool[cite: 38].
  - $$\forall t_1, t_2, p_1, p_2, a_1, a_2 : \text{Taxpayer}(t_1) \land \text{Taxpayer}(t_2) \land \text{married}(t_1, t_2, \text{True}) \land \text{married}(t_2, t_1, \text{True}) \land \text{get\_pool}(t_1, p_1) \land \text{get\_pool}(t_2, p_2) \land \text{get\_allowance}(t_1, a_1) \land \text{get\_allowance}(t_2, a_2) \implies p_1 = p_2 \land (a_1 + a_2) = p_1$$

- **Allowance Logic Consistency**: A taxpayer's allowance must match their status (Age, Lottery). We define $E(t)$ as the expected allowance for taxpayer t:

$$E(t) = \begin{cases} 9000 \geq \text{if } t_{\text{is\_winner}} \\ 7000 \quad \text{if age}(t) \geq 65 \\ 5000 \quad \text{otherwise} \end{cases}$$

  The invariant requires that for any taxpayer t, their stored pool allowance $P(t)$ matches the expected value (summed if married)[cite: 37, 41, 45].
  - $\forall t : \text{Taxpayer}(t) \implies \text{get\_pool}(t) = (E(t) + (\text{is\_married}(t)?\, E(\text{spouse}(t)) : 0))$

2. The Lottery Problem

For the lottery system (Part 4), we must ensure fairness and correct state transitions. The lottery allows users under 65 to win a higher tax allowance.

- **Lottery Eligibility**: Participants must be valid taxpayers and under the age of 65.
  - $\forall t : \text{Participating}(t) \implies \text{Taxpayer}(t) \land \text{age}(t) < 65$

- **Conservation of Winners**: The number of winners must be consistent with the number of completed lotteries. A participant cannot win multiple times arbitrarily without a new lottery round.

- **Unbiased Distribution**: Over a large number of runs N, the frequency of user t winning should be roughly $\frac{1}{|T|}$ where $|T|$ is the participant count.
  - $$\text{count\_wins}(t) \approx \frac{N}{\text{count\_participants}}$$

3. The Pattern Problem

Beyond logical invariants, the contract was inspected for structural security vulnerabilities.

- **The age problem**:

First of all storing the age of the customer with the year alone, may not be that bad, the problem is that we need to create a routine, something akin to a cron-job which will call the "haveBirthday()" function once a year, furthermore the function needed some railguards to prevent it from being called more than once each year.

A Taxpayer could call this function until the age would surpass 65 therefore receiving more allowance for his taxes. This is obviously wrong, so we chose to make the age of the Taxpayer a unixtimestamp(int256) this allows us to know exactly when the user has his birthday. We then can calculate the year of the Taxpayer by using the "getYearsSinceBirth()" function, this prevents fraudolent behaviour from the users, we then need to account for the raising of the allowance once they reach 65 years. The best thing to do would be to use a cron-job or something similiar.

For this project we chose to do something simpler, the Taxpayer himself is responsible for calling the function "raiseOwnAllowance()" which of course follows the CEI pattern, and flags when the user correctly passes the checks, therefore he cannot call it more than once in his life.

- **Check-Effects-Interactions (CEI)**: We identified that the original code did not strictly follow the CEI pattern. To prevent potential re-entrancy attacks during external calls (e.g., transfers), we refactored the functions to update state variables **before** interacting with external addresses.

This can be seen in the marriage function, transfer allowance, and many others, in many of these:

1. Some checks were missing
2. Some effects were not happening before the interactions
3. And some interactions were missing.

For example if I marry, my spouse should immediately marry me as well, so the solution is to make another function, “marry_me()” that will be called after I set them as my spouse. This and some checks inside of the marry_me functions ensure us atomicity.

- **Access Restriction**: We saw how the code lacks even the most basic access restrictions, for example the lottery could be ended by any address inside of the block-chain,by implementing the onlyBy modifier we can specify which address can call which functions, in this case the owner.

This problem can also be found in the Taxpayer contract since when we want to marry or when we want to divorce anyone could call this functions, this is solved in the same manner.

- **Method Verification**: Problem, How do we assure that when we marry the address of the spouse we provide actually points to a Legitimate Taxpayer contract?

The first thing that comes to mind is the ERC165 by implementing this we could ask the address which methods it implements, but this is unreliable since we could marry a FAKE Taxpayer contract which is very dangerous.

We found that the more reliable thing to do would be that of actually hashing the code of the contract and comparing the spouse before actually marring. However since the Taxpayer contract in our implementation actually has immutable values this is the wrong approach. So the solution is to create a Taxpayer Factory which keeps a mapping of each and every taxpayer it has created, so that all the Taxpayer needs to do is to check if the spouse’s address is in the mapping. This can be useful for the lottery as well. For simplicity’s sake and for the fact that the requirements did not actually tell us if the user could participate in multiple lotteries, we chose to limit the number of simultaneous lotteries shall be one. This can be easily done by creating a State contract which is the factory for the taxpayers and the single lottery, this lottery will be reusable, and we write pre-conditions such that it cannot be called and we also follow the C.E.I. so that we prevent any possible re-entrant attack.

4. Code

```
require(startTime == 0);
```

4.1. test lottery

```
function proxy_player_commit(uint256 playerIndex) internal {
      uint256 idx = playerIndex % NUM_PLAYERS;
      players[idx].joinLottery();
  }

  function player_round(uint256 idx) internal {
      proxy_player_commit(idx);
  }

  function lottery_rounds() internal {
      for (uint256 x = 0; x < N_OF_ROUNDS; x++) {
          s.proxy_startlottery();

          for (uint256 index = 0; index < NUM_PLAYERS; index++) {
              player_round(index);
          }

          t.test_vesting(1 days); // NOTE: This makes time pass by one day
          s.proxy_endlottery();

          t.test_blocks_forward(2); //NOTE: Makes the blocks go forward by one
      }
  }

  function safe_exp_value(uint256 player_idx) internal {
      Taxpayer p = players[player_idx];
      int256 exp_val = int256((N_OF_ROUNDS + 1) * (10 ^ 18 / NUM_PLAYERS));
      int256 approx_exp_val = int256(p.getLotteryWins() * 10 ^ 18);
      int256 delta = exp_val - approx_exp_val;

      if (delta < 0) {
          delta = -delta;
      }

      if (delta > 5 * 10 ^ 18) {
          emit AssertionFailed(string.concat(
                  "The lottery is unfair, Expected val:",
                  Strings.toStringSigned(delta),
                  " Approx:",
                  Strings.toStringSigned(approx_exp_val),
                  " Delta:",
                  Strings.toStringSigned(delta)
              ));
      }
  }

  function invariant_test_fairness() public {
      lottery_rounds();

      for (uint256 index = 0; index < NUM_PLAYERS; index++) {
          safe_exp_value(index);
      }
  }
```

4.2. test taxpayer

```
function forEach(function(Taxpayer) internal constraint) internal {
        for (uint256 index = 0; index < N_OF_TAXPAYER; index++) {
            constraint(taxpayer[index]);
        }
}
```

Listing 4: ∀ utility.

```
    function checkMarried(Taxpayer t1) internal {
           Taxpayer spouse = t1.get_spouse();
           if (address(spouse) == address(0)) return;

           if (address(spouse.get_spouse()) != address(t1)) {
               emit AssertionFailed("The spouse is different");
           }
       }

       function invariant_are_both_married() public {
           forEach(checkMarried);
       }
```

Listing 5: Check Married Invariant this checks the $\text{me\_married\_with\_spouse} \iff \text{spouse\_married\_with\_me}$ constraint.

```
    function checkAllowance(Taxpayer t1) internal {
        if (t1.getTaxAllowance() > t1.getPoolAllowance()) {
            emit AssertionFailed("Too much money saved in taxes");
        }
        Taxpayer sp = t1.get_spouse();

        if (address(sp) != address(0)) {
            if (t1.getPoolAllowance() != sp.getPoolAllowance()) {
                emit AssertionFailed("The pool allowance must be equal for both
spouses");
            }
            if ((t1.getTaxAllowance() + sp.getTaxAllowance()) !=
t1.getPoolAllowance()) {
                emit AssertionFailed("Too much money saved in taxes naughty
couple");
            }
        }
    }

    function invariant_is_tax_allowance_good() public {
        forEach(checkAllowance);
    }
```

Listing 6:

```
function checkAgeAllowance(Taxpayer t1) internal {
        uint256 my_mod = 0;
        uint256 my_num = 5000;
        if (t1.getYearsSinceBirth() ⩾ oldAge && t1.get_counter()) {
            my_mod += 2000;
        }
        my_mod += (t1.getLotteryWins() * 2000);
        Taxpayer sp = t1.get_spouse();
        if (address(sp) ≠ address(0)) {
            my_num = my_num * 2;
            if (sp.getYearsSinceBirth() ⩾ oldAge && sp.get_counter()) {
                my_mod += 2000;
            }

            my_mod += (sp.getLotteryWins() * 2000);
        }
        if (t1.getPoolAllowance() ≠ (my_num + my_mod)) {
            emit AssertionFailed(string.concat("The poolTaxAllowance is wrong",
Strings.toString(my_num + my_mod)));
        }
    }

    function invariant_is_aged_tax_allowance_good() public {
        forEach(checkAgeAllowance);
    }
```

Listing 7: This checks the allowance counting the age and the lottery wins.

4.3. Conclusions

Using Echidna, we successfully verified the core invariants of the `Taxpayer` system. The initial fuzzing campaign revealed violations in the marriage logic (one-way marriage bugs) and tax pooling calculations, which were resolved by enforcing the bidirectional constraints described in Section 2. While for the lottery we managed to re-write the code base making it impossible to instantiate the lottery more than once and making a person only win once and not more times at the same time.